

Case Study Hot Delivery Report

Executive Summary

This report addresses the operational challenges of the "profitability paradox" in the online food delivery sector, where explosive market growth is frequently offset by high driver compensation costs. We developed and evaluated a series of prescriptive models, ranging from mixed-integer linear programs (MILP) to scalable heuristics, to optimize driver-to-order assignments and identify key strategic trade-offs between cost efficiency and service quality.

Below details the key insights by operational assumption:

- **Single Driver Baseline:** We established a theoretical lower bound for operational costs by minimizing total distance. This "distance-only" scenario achieves absolute minimum fuel and labour costs but significantly sacrifices customer satisfaction, as it lacks temporal constraints to prevent delivery delays.
- **Temporal Constraints:** Introducing wait-time thresholds W reveals that the trade-off between distance and speed is not a steady slope but a step function. We identified that average-wait metrics can hide "service disasters", namely extreme individual delays, hence recommending a maximum-wait formulation to guarantee a consistent service floor for all customers.
- **Multi-Driver Assignment:** Generalizing to multiple drivers highlights the diminishing marginal returns of labour. While a second driver significantly improves both distance and speed, a third driver often remains idle unless service requirements are extremely tight ($W \leq 10$), mirroring the logic behind real-world surge pricing.
- **Scalable Heuristics:** To support real-time dispatching, we designed a "Cluster-Then-Route" greedy heuristic. Although this approach results in 36.6% more distance than the global MILP optimal, it solves over 1,000x faster on the recorded benchmark run, making it the only viable option for high-velocity operational environments.

Following the analysis, we recommend that to balance profitability with brand reputation, platforms should adopt maximum-wait constraints rather than system averages. Furthermore, the deployment of online heuristics is essential for real-time responsiveness to driver cancellations and new order arrivals.

Introduction

Business Context and Problem Overview

Despite the explosive growth of platforms like Uber Eats and DoorDash, achieving profitability remains a challenge due to high operational costs and narrow margins. The most significant expense is driver compensation, which is primarily determined by distance. To address this, online food delivery companies rely heavily on prescriptive routing models to minimize travel and reduce costs. As consultants for Uber Eats, we have developed a series of mixed-integer

linear programs (MILP) to optimize driver-to-order assignments and identify the most efficient delivery routes.

Project Scope and Methodology

Our analysis follows an incremental framework to evaluate strategic and operational trade-offs, solved using the PuLP and Gurobi libraries. The report is organized into the following four parts:

- **Part I Single-Driver Baseline:** Designing an optimal route for one driver to minimize total distance for pre-assigned orders.
- **Part II Temporal Constraints:** Incorporating restaurant preparation times and customer waiting limits W to analyze the sensitivity of distance relative to service speed.
- **Part III Multi-Driver Assignment:** Generalizing the model to assign and route orders across a fleet of drivers with different starting locations and transport modes.
- **Part IV Scalable Heuristics:** Proposing a simplified, scalable solution for large-scale, real-time operational environments where traditional optimization may be computationally prohibitive.

Part I

Part I establishes the foundation of our routing engine using a Flow-Based Mixed-Integer Program (MIP). The primary objective is to minimize total distance traversed, which serves as the theoretical lower bound for operational costs (driver compensation).

The model is governed by the following key constraints:

- **Network Integrity:** Ensures flow conservation across consecutive arcs and prohibits self-loops.
- **Service Requirements:** Mandates that every restaurant and neighborhood node is visited exactly once.
- **Operational Logic:** Requires the driver to start at Downtown Toronto (Rosedale) and finish at the final delivery neighborhood.
- **Precedence:** Enforces "Pick-up before Delivery" logic for all assigned orders.

For the mathematical expression of the model, its objective function and constraints, please refer to Appendix - Part I Model Structure.

Prescribed Routes and Results

The model successfully identified the global optimum for both datasets in approximately one second, reflecting the efficiency of the baseline formulation.

Instance	Total Distance	Route
A	4.7km	Rosedale -> Underground City -> Central Bay Street

B	33.9km	Rosedale -> Central Bay Street (2 stops) -> R/A/K (2 stops) -> Underground City -> S/C -> York (Cedervale) -> Central Toronto (A/NM/Y) -> Etobicoke Northwest -> Etobicoke
----------	--------	--

Analysis

Part I represents an idealized "distance-only" scenario. By ignoring temporal variables, the model achieves the absolute minimum fuel and labor cost. However, this efficiency comes at the expense of customer service. Without a time constraint, the model may prioritize a shorter total path that significantly delays early orders, a trade-off between operational cost-savings and customer satisfaction that we address in Part II.

Part II

Part I treated the problem as a single-driver routing problem and minimized total distance. Part II keeps that structure but adds the constraint that matters most in food delivery: time. Orders are not available immediately. Food can sit after it is ready. Customers can wait too long even when the route is short. The case therefore keeps the same objective and adds a service requirement on top of it.

The model changes in four ways. Each order now has a ready time. The route must track when the driver reaches each stop. Customer waiting time is defined as the time between food readiness and arrival at the customer's neighborhood. The route must also satisfy a waiting-time threshold W . We keep the same single-driver setting from Part I: the driver starts in Rosedale, can carry multiple orders, and does not need to return to the starting point. The case also adds a five-minute service time at the customer location, which affects the timing of later stops. The full formulation is left to the appendix. In the main text, the point is simple: Part II turns the Part I route into a time-feasible route.

We use $W = 60$ as the benchmark setting for the main discussion because it gives one loose instance and one binding instance.

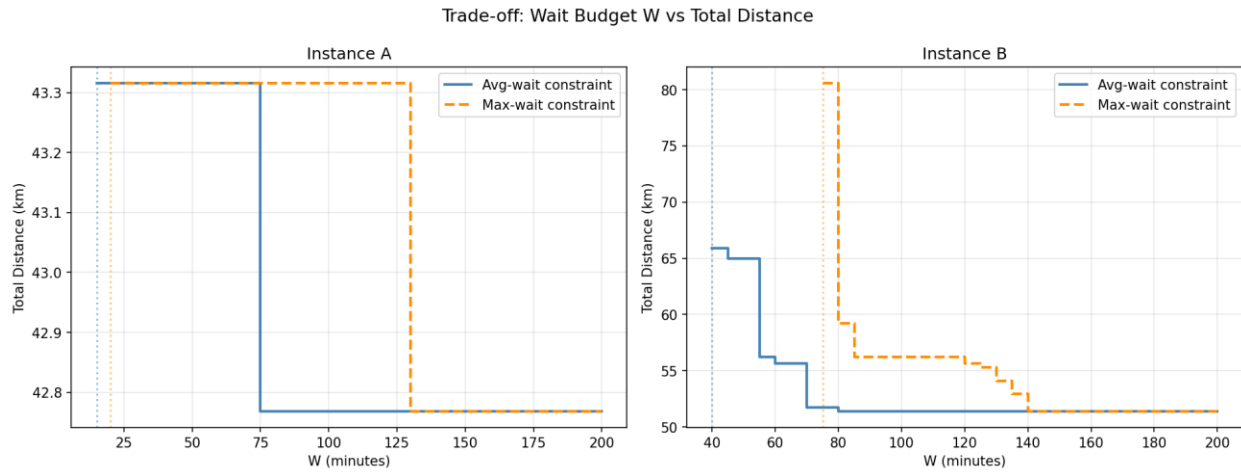
Analysis block	Instance	W	Formulation	Distance (km)	Average wait (min)	Maximum wait (min)
Benchmark	A	60	Average wait	43.3158	12.32	15.51
Benchmark	B	60	Average wait	55.6285	58.40	115.12
Alternative-metric comparison	B	90	Average wait	51.4020	78.56	137.35
Alternative-metric comparison	B	90	Maximum wait	56.2080	52.43	83.99

This table sets up the rest of the section. Instance A is comfortably feasible at $W = 60$. Its average wait is low, and its maximum wait is also low. Instance B is much tighter. Its average wait is 58.40 minutes, already close to the 60-minute limit, while the worst-served order waits 115.12 minutes. That contrast matters. Instance A shows what the model looks like when the service rule is loose. Instance B shows what happens when time starts to control route choice.

The waiting-time threshold W is the main lever in this part. When W gets smaller, the model becomes harder because fewer routes remain feasible. A route can be short and still be rejected if

it delays delivery too long after food is ready. Tightening W therefore removes routes that would otherwise be attractive in distance terms. Relaxing W does the opposite. It gives the route more flexibility and can make shorter routes feasible again. That is the real difficulty shift in Part II: the objective stays the same, but the feasible set changes.

The feasibility thresholds make this visible. Under the average-wait formulation, Instance A first becomes feasible at $W = 15$ and Instance B at $W = 40$. Under the maximum-wait formulation, the first feasible thresholds rise to $W = 20$ for Instance A and $W = 75$ for Instance B. The pattern is clear. The maximum-wait version is stricter, and Instance B is the harder case. A tighter service promise does not just make the problem more constrained. It rules out specific route structures.



This figure should be the only main-body figure in this section because it does real work. It explains both route sensitivity and curve shape. The distance- W relationship is not smooth. It moves in steps. That shape comes from feasibility, not from gradual adjustment. Over some ranges of W , the same route remains optimal, so distance stays flat. Then W crosses a threshold, a previously infeasible route becomes feasible, and distance drops at once. The trade-off is real, but it appears at route-switch points rather than in a clean line.

That shape also helps with interpretation. A tighter service threshold does not always increase distance immediately. Sometimes the current route survives a modest tightening with no cost change. But once the threshold cuts off that route, the model has to jump to a worse one. This is the useful managerial point in the trade-off curve: service quality has a routing cost, but that cost often appears in thresholds rather than in a steady slope.

The average-wait constraint creates a second problem. It can hide a very bad customer experience inside a reasonable system average. Instance B at $W = 60$ shows this directly. The route is feasible because the average waiting time is 58.40 minutes. But the maximum waiting time is 115.12 minutes. One order is clearly doing much worse than the average suggests. Average wait measures system performance. It does not protect the worst-served order.

To address that weakness, we tested a maximum-wait formulation. The objective did not change. We still minimized total distance. The only change was the service rule. Instead of limiting the average wait across all orders, we limited the worst individual wait. This gives a cleaner test of service fairness because it removes the ability to offset one bad order with several good ones.

At $W = 90$ in Instance B, the average-wait formulation gives a shorter route at 51.4020 km, but it still leaves a maximum wait of 137.35 minutes. The maximum-wait formulation raises distance to 56.2080 km, lowers the average wait to 52.43 minutes, and caps the worst wait at 83.99 minutes. This is not a cosmetic change. It is the operating cost of guaranteeing a service floor to every order.

Part II therefore does more than add time to Part I. It shows that service quality is not just a reporting metric placed after the route is found. It changes which route the model is willing to choose. That insight matters again in Part III, where the same tension between efficiency and service quality will shape both routing and assignment.

Part III

We now extend our model to account for multiple drivers in the delivery problem. The model changes in the following ways:

(1) Driver index d added to all variables:

New driver index d (less or equal to D the maximum number of drivers to be deployed) is added to all variables, so every variable now carries a driver index, making the problem simultaneously route and assign.

(2) Order assignment variable a_{dk} and driver coverage constraint:

Each order needs to be picked up and delivered by the same driver, represented by the new assignment variable a_{dk} : binary; 1 if driver d is assigned order k . It links the routing sub-problem per driver to a global coverage requirement. Accordingly, an additional coverage constraint requires every order k to be served by exactly one driver: $\sum_d a_{dk} = 1$.

(3) Per-driver depot and speed:

Each driver d has its own depot node and speed, so the node graph is replicated D times. The depot indices differ across drivers even though restaurant/neighbourhood nodes are shared logically, and the travel time $\tau_{ij}^d = 60c_{ij}/v_d$ now depends on the driver's modal (walk/bicycle/car), making wait-time computations driver-specific.

(4) Big-M per driver for per-driver arrival time constraint:

M_d is now driver-specific because each driver's worst-case arrival time depends on their speed.

(5) Deployment indicator y_d :

This is a binary variable taking value 1 if driver d departs. Including it allows the solver to endogenously choose how many drivers to use, which supports the assignment purpose in Part III and the scalability discussion in Part IV.

The detailed model structure with the variable definition, objective function and all constraints listed is shown in the Appendix – Part III. Next, we'll discuss our findings after applying the model on part3_small.csv. We apply two versions of the model: version 1 with rigid D

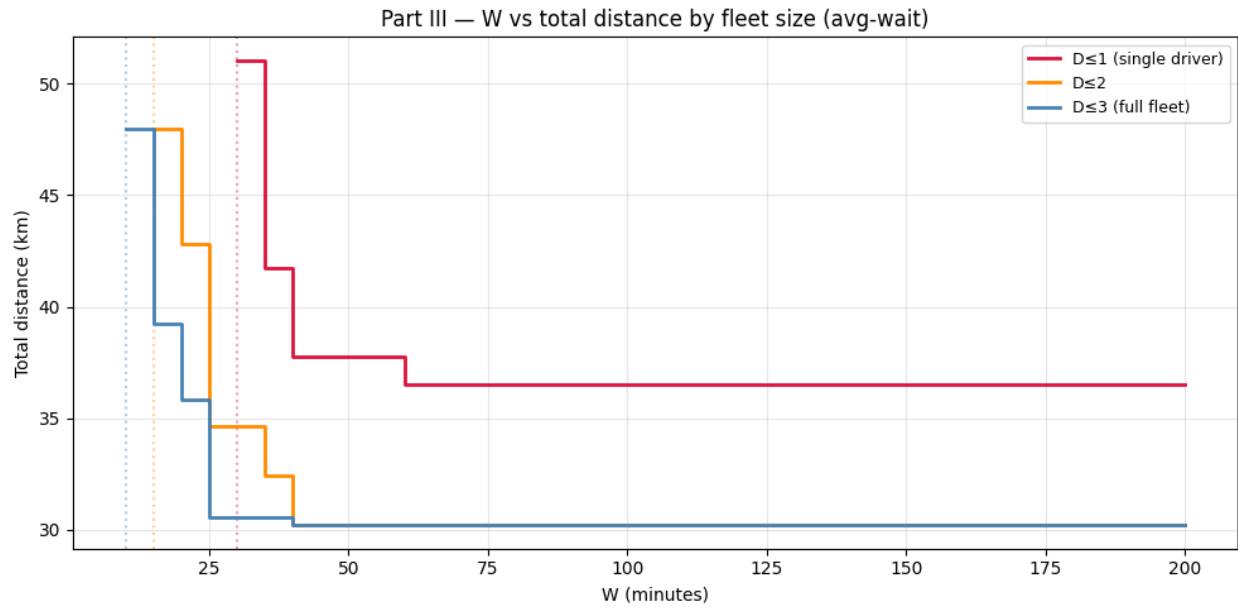
(maximum number of drivers deployable) looping from 1 to 3 and W to demonstrate the trade-off between fleet size and service responsiveness, and version 2 with the model internalizing the decision of the optimal assignment of drivers given the driver pool.

Solution

Trade-off between driver availability and service responsiveness

Driver Cap	(W)	Distance (km)	Avg Wait	Max Wait	Drivers Used	Note	Solver Run Time (s)
D≤1	30	51.0649	30.00	61.49	[0]		90.1
	35	41.7501	35.00	107.31	[0]	−9.31 km	66.0
	40	37.7396	36.62	102.71	[0]	−4.01 km	14.7
	45	37.7396	45.00	113.20	[0]		15.5
	50	37.7396	36.62	102.71	[0]		19.4
	60	36.5282	57.93	118.62	[1]	−1.21 km	17.7
	75	36.5282	75.00	139.96	[1]		18.8
	90	36.5282	90.00	172.08	[1]		14.7
	120	36.5282	120.00	222.08	[1]		13.4
	150	36.5282	150.00	272.08	[1]		8.5
	200	36.5282	57.93	118.62	[1]		11.4
D≤2	15	47.9665	15.00	20.34	[0, 1]		122.2
	20	42.8272	20.00	61.46	[0, 1]	−5.14 km	123.0
	25	34.6293	25.00	90.90	[0, 1]	−8.20 km	14.3
	30	34.6293	27.16	94.00	[0, 1]		31.0
	35	32.4118	34.46	91.24	[1, 2]	−2.22 km	15.0
	40	30.1836	35.38	100.14	[0, 1]	−2.23 km	23.9
	45	30.1836	35.38	100.14	[0, 1]		18.7
	50	30.1836	35.38	100.14	[0, 1]		19.2
	60	30.1836	35.38	100.14	[0, 1]		23.0
	75	30.1836	35.38	100.14	[0, 1]		10.1
	90	30.1836	35.38	100.14	[0, 1]		7.0
	120	30.1836	35.38	100.14	[0, 1]		5.6
	150	30.1836	35.38	100.14	[0, 1]		5.2

	200	30.1836	35.38	100.14	[0, 1]		5.1
D≤3	10	47.9580	9.73	17.80	[0, 1, 2]		121.7
	15	39.2623	12.86	31.27	[0, 1, 2]	−8.70 km	36.6
	20	35.8253	19.75	52.22	[0, 1, 2]	−3.44 km	18.9
	25	30.5441	23.29	85.99	[0, 1, 2]	−5.28 km	5.1
	30	30.5441	23.29	85.99	[0, 1, 2]		8.0
	35	30.5441	35.00	144.53	[0, 1, 2]		8.6
	40	30.1836	35.38	100.14	[0, 1]	−0.36 km	7.0
	45	30.1836	35.38	100.14	[0, 1]		8.8
	50	30.1836	35.38	100.14	[0, 1]		9.1
	60	30.1836	35.38	100.14	[0, 1]		7.0
	75	30.1836	35.38	100.14	[0, 1]		6.9
	90	30.1836	44.49	100.14	[0, 1]		7.5
	120	30.1836	35.38	100.14	[0, 1]		6.2
	150	30.1836	35.38	100.14	[0, 1]		5.9
	200	30.1836	35.38	100.14	[0, 1]		6.2



Optimal Solution after Internalizing Driver Assignment at Required W = 60

Number of Orders	Assigned Drivers	Total Distance (km)	Avg Wait (min)	Max Wait (min)
5	[0, 1]	30.18	35.38	100.14

Order ID	0	1	2	3	4
Wait Time (min)	100.14	38.55	24.35	12.37	1.47
$\leq W?$	X	✓	✓	✓	✓

The distribution of waiting times is highly skewed, with order 0 experiencing a wait of 100.14 minutes — more than double the next longest wait. The full arc-by-arc routing solution that produces this distribution is shown below:

The Routing Solution

Driver 0 (Speed: 40km/h, Depot: DT (R/A/K))

Step	Move	From	To	Distance (km)	Arrival Time (min)
1	Depot→R4	DT (R/A/K)	DT (K/C/G)	1.27	7.00
2	R4→R0	DT (K/C/G)	DT (Central Bay)	0.98	8.47
3	R0→N4	DT (Central Bay)	DT (Central Bay)	0.00	8.47
4	N4→R2	DT (Central Bay)	DT (Ryerson)	0.63	60.00
5	R2→R1	DT (Ryerson)	DT (Christie)	3.61	65.42
6	R1→N1	DT (Christie)	West Toronto (B/P/E)	3.42	70.55
7	N1→N2	West Toronto (B/P/E)	York (F/O)	5.87	84.35
8	N2→N0	York (F/O)	North York (A/W/D)	7.19	100.14

Driver 1 (Speed: 35km/h, Depot: DT (St. James Park))

Step	Move	From	To	Distance (km)	Arrival Time (min)
1	Depot→R3	DT (St. James Park)	DT (St. James Park)	0.00	36.00
2	R3→Depot	DT (St. James Park)	DT (St. James Park)	0.00	36.00
3	Depot→N3	DT (St. James Park)	East Toronto (The Beaches)	7.22	48.37

Driver 2 (Speed: 32km/h, Depot: DT (Church and Wellesley)) – No assigned tasks

The table shows why order 0 waits 100.14 minutes: Driver 0 must first complete orders 4, 2, and 1 before reaching order 0's neighbourhood (North York A/W/D), arriving at minute 100.14. The release time for order 0's restaurant is minute 8.47, so the food sits waiting for 91.67 minutes of that total. This is the concrete evidence for the claim in Q3 that average-wait metrics can mask severe individual outcomes.

Analysis

Q1. How much harder does the problem become as W changes?

As W decreases from 200 to 10 minutes, the average-wait constraint tightens, reducing the feasible solution space. First, total distance increases stepwise from 30.18 km to 47.96 km, as the optimal solution remains unchanged over intervals of W and shifts only when it becomes infeasible. Second, solver runtime rises sharply from seconds to several minutes, as distance-minimizing assignments increasingly violate the wait constraint and require extensive backtracking. Third, for $W > 40$ minutes, the constraint becomes non-binding and the optimal solution stabilizes at 30.18 km, indicating a saturation regime. Computational difficulty peaks

near the feasibility boundary around $W = 10-20$, rather than in clearly infeasible or loosely constrained regions.

Q2. Trade-off curve shape and correlation between W and distance

As shown from left diagram in Q1, the trade-off curve is a step function. Distance decreases as W relaxes because with more temporal slack the solver can consolidate orders onto fewer drivers or exploit geographic proximity without worrying about wait-time deadlines. Once W reaches the natural wait time of the unconstrained optimum at around 40 min for avg-wait, relaxing W further has zero marginal effect, and the curve plateaus at 30.18km, representing the pure distance-minimizing solution unconstrained by time.

On correlation, W and total distance are negatively correlated in the constrained region ($W = 10-40$), and effectively uncorrelated in the plateau region $W \geq 40$. The elasticity of distance with respect to W is non-constant: high near the constraint boundary, and zero in the unconstrained zone. This asymmetry is managerially important: small improvements in service time near the infeasible edge are very expensive in distance, while moderate service standards $W \geq 40$ come at no extra cost at all.

Avg vs max metric: The max-wait curve saturates later at about $W \approx 110$ and has a higher baseline distance. This is because max-wait constrains the worst-off order specifically, requiring the solver to add a detour for one outlier order rather than averaging it away. Avg-wait allows the outlier to persist as long as the mean is acceptable, which is structurally more lenient.

Q3. Problem with avg waiting time and alternative metric performance

W	Objective	Distance (km)	Avg Wait (min)	Max Wait (min)
60	Avg-wait	30.18	35.38	100.14
60	Max-wait	35.16	23.04	57.03
90	Avg-wait	30.18	35.38	100.14
90	Max-wait	30.54	23.29	85.99
120	Avg-wait	30.18	35.38	100.14
120	Max-wait	30.18	35.38	100.14

As shown by the above result table, using average waiting time as the objective metric masks extreme individual outcomes. At $W = 60$ under avg-wait, the average is only 35.38 minutes, but the maximum is 100.14 minutes. The solver legally satisfies the average constraint while still leaving one order with a very poor outcome. In practice this is a service-quality problem: one customer gets a cold meal and a one-star review regardless of what the average customer experiences.

Constraining the maximum guarantees a service floor for every individual customer. At $W = 60$ under max-wait, the maximum drops to 57.03 minutes (vs 100.14 under avg-wait), but at the cost of 16.5% more distance (35.16 vs 30.18 km). This is the equity-efficiency trade-off in operations: protecting the worst-case customer is more expensive, but more robust to complaints and churn. It mirrors the difference between utilitarian and egalitarian optimization objectives in resource allocation. Uber Eats' actual guarantee SLAs (for example, "delivered within X

minutes") are closer to max-wait logic than avg-wait logic, because a single extreme delay generates reputational damage disproportionate to its weight in an average.

Q4. Benefits and disadvantages of fewer drivers ($D = 1,2,3$; $W = 5,10,30,60,120$)

From the trade-off between driver availability and service responsiveness table in solution session, at $W = 60$, going from $D=1$ to $D=2$ yields a 17.4% reduction in total distance (36.53 to 30.18 km) and a 38.9% reduction in average wait time (57.93 to 35.38 minutes). This is the most impactful marginal driver. The single driver in $D=1$ must handle all 5 orders sequentially, accumulating spatial backtracking and wait-time pressure. The second driver, faster and from a different depot, can claim geographically proximate orders, eliminating cross-city detours. Moving from $D=2$ to $D=3$ gives no improvement, shown by the non-deployment of the third low-speed driver in both cases. This occurs because the 2-driver solution already reaches the unconstrained distance minimum, so the third driver offers no additional geographic or temporal benefit. The marginal cost of deploying a third driver (labor, insurance, idle time) would be strictly wasteful here.

For robustness, we also tested tighter constraints at $W = 5, 10$, and 30 to examine whether adding more drivers improves feasibility when the wait-time limit is more extreme. At $W = 5$, the problem is infeasible for all fleet sizes regardless of driver count. At $W = 10$, $D \leq 1$ and $D \leq 2$ remain infeasible, but $D \leq 3$ becomes feasible at 47.96 km and an average wait of 9.73 minutes, so the third driver is not merely convenient here but the marginal resource that unlocks feasibility entirely. At $W = 30$, all fleet sizes are feasible but with meaningful differences: $D \leq 1$ requires 51.06 km with average wait binding at 30.00 minutes, $D \leq 2$ improves to 34.6 km and 27.2 minutes, and $D \leq 3$ achieves 30.5 km and 23.3 minutes, with each additional driver yielding a substantial distinct improvement.

The disadvantages of fewer drivers ($D=1$ vs $D=2$) include higher total distance as the single driver must traverse more of the city; higher average and maximum wait times as sequential routing means later orders wait for all prior ones; tighter feasibility since $W = 60$ is barely satisfied at $D=1$ (avg=57.93 min) leaving no buffer for real-world delays; and no redundancy, as a single driver delay or cancellation breaks all 5 orders simultaneously. The advantages of fewer drivers include lower labor cost since fewer active drivers means lower platform commission payouts; simpler coordination since single-driver assignment requires no inter-driver synchronization; and potentially higher driver earnings per trip from more orders per route.

Q5. How do these insights relate to current Uber Eats and DoorDash driver strategies?

Dynamic surge pricing and the saturation point: The finding that $D=3$ provides no benefit over $D=2$ at $W = 60$ but becomes the critical marginal driver at $W = 10$ directly mirrors how platforms regulate driver supply through surge pricing. The saturation point is not fixed. It shifts with the effective service-level requirement. When order density is low and wait-time budgets are loose, platforms lower the incentive multiplier to avoid idle drivers, effectively implementing the saturation logic the model reveals. Conversely, surge pricing during peak hours is a signal that the system is operating in the tight- W regime, where the marginal driver genuinely unlocks feasibility rather than merely improving efficiency. Our result that $D=3$ is essential below $W \approx$

10 but wasteful above $W \approx 30$ suggests a possible mechanism by which tighter service guarantees could justify higher driver incentives — consistent with, though not a direct model of, surge pricing logic.

Batching and the average vs. max wait trade-off: The $D=1$ solution shows a single driver serving all 5 orders in one route, which is the stacking strategy Uber Eats and DoorDash actively incentivize. Batching reduces employment cost compared to dispatching separate drivers per order when the W constraint is slack $W \geq 60$. However, the $W = 30$ result from Q3, where $D \leq 2$ produces a maximum wait of 94 minutes despite satisfying the average constraint, shows this comes at the cost of severe worst-case outcomes for individual customers. Our $W=30$ result — where $D \leq 2$ produces a maximum wait of 94 minutes despite satisfying the average constraint — provides a possible structural explanation for documented complaints about stacked orders: a platform optimizing an average metric would legally satisfy its own routing constraint while still producing extreme individual delays.

Driver positioning and depot proximity: Across all W values tested, the solver consistently selects Driver 0 first, and Driver 1 second. Driver 2, who is slower and more peripherally located, is only deployed at $W = 10$ where feasibility demands it. This mirrors Uber Eats' hotspot strategy, where drivers are nudged to position in high-density restaurant zones. The platform's routing model implicitly values depot proximity to restaurant clusters, and the velocity heterogeneity across drivers explains why platforms offer different incentives to cyclists, pedestrians, and car drivers operating in the same zone.

Average vs. max wait in platform SLAs: The Q3 analysis mirrors Uber Eats' delivery time guarantee feature: the platform promises arrival within a specific window to individual customers, not just an average. This is max-wait logic in practice. The 16.5% distance premium the model reveals at $W = 60$ under max-wait versus avg-wait is the real operational cost of maintaining that per-customer guarantee, which is a cost platforms must recoup through higher delivery fees or reduced driver margins. The $W = 30$ result makes this even starker: satisfying the average constraint allows a maximum wait of 94 minutes, which would constitute a clear SLA breach under any individual guarantee.

Why platforms lose money: The $W \times D$ results formalize the core trade-off: minimizing distance (cost) conflicts with minimizing wait time (service quality), governed by constraint tightness W and fleet size (D). Our model suggests that service-focused strategies, where platforms accept higher routing costs to ensure speed, resemble a low- W , high- D operating regime. Cost-focused strategies that rely on batching and fewer active drivers resemble a high- W , low- D regime. Whether specific platforms like DoorDash or Uber Eats fully match either profile is beyond what our small-instance model can confirm — but the structural trade-off the model reveals is consistent with publicly documented differences in their driver deployment strategies. Intense fee competition prevents cost recovery, explaining persistent unprofitability despite high revenues within the industry.

With Part III set up, the problem now becomes a vehicle routing problem with time windows, and the size explosion is quadratic in drivers and cubic overall. This justifies why scaling becomes the focus of Part IV.

Part IV. Scalable Heuristic Model for Operation

In this part, the scalability issue that presented itself in Part III is addressed with a quick-to-run and simple heuristic that scales well to large-scale problems.

Cluster-Then-Route Greedy Heuristic

From Part III, it became evident that drivers cluster by geography. The optimal solution never assigned a driver to an order that was geographically far from their starting location when a closer driver was available. In fact, routes within each driver’s cluster sweep geographically without backtracking. This was illustrated in the Part III small instance when Driver 2 (whose depot is St. James Park) was assigned Order 4 (whose restaurant is St. James Park). In this example, there is zero depot-to-restaurant distance. On the other hand, Driver 1 (whose depot is Richmond/Adelaide) handled the four downtown-clustered orders since they were already located downtown. This finding naturally motivates a heuristic with three natural phases: cluster, route, and repair.

The model changes from Part III in the following key ways:

(1) Replacing the MILP with a greedy procedure

Part III solves a single mixed-integer program that simultaneously optimizes assignment and routing across all drivers and all orders. For the large instance (15 orders, 10 drivers), this would require 288,300 binary variables, which is computationally infeasible within the time limits required for real dispatch operations. The heuristic replaces the global optimization with a sequential greedy procedure by assigning first, then routing, then repairing. Each step is fast and interpretable at the cost of not guaranteeing the globally optimal solution.

(2) Decomposing the problem into three independent sub-problems

Rather than solving assignment and routing jointly, the heuristic decomposes the problem into three phases that are solved in sequence. Phase 1 handles assignment, Phase 2 handles routing, and Phase 3 handles feasibility. This decomposition is what makes the method scalable: each phase runs in polynomial time whereas the joint MILP is NP-hard.

(3) Replacing the time-step structure with a time-clock structure

The Part III MILP tracks time through discrete arc slots $t = 0, 1, \dots, T-1$. The heuristic instead maintains a continuous clock $A_d(v)$, the driver’s actual arrival time at each node, which accumulates as they move through their route. This is simpler to implement and to compute, and it naturally handles the food-release wait and 5-minute customer dwell without requiring big-M linearization.

(4) Adding an explicit feasibility repair phase

The Part III MILP enforces the average wait constraint W as a hard constraint within the optimization. Because the heuristic’s greedy assignment may violate this constraint, a dedicated repair phase is added: if the average wait exceeds W after routing, the heuristic iteratively re-assigns the worst-offending order to a different driver until the constraint is satisfied or no improving transfer exists. This is a local-search step that the MILP does not need.

Phase 1 – Clustering

Orders are processed in ascending food-release-time order. This means that the most time-sensitive orders are assigned first, so they get first pick of the nearest available driver. For each order k , it is assigned to the driver d^* whose depot is the closest to that order's restaurant:

$$d^* = \arg \min_d c(\text{depot}_d, R_k)$$

This directly replicates the geographic clustering observed in the Part III MILP solutions.

Phase 2 – Routing (Nearest-Neighbour per Driver)

Once all orders are assigned, each driver independently sequences their stops using a nearest-neighbour greedy rule. At every step, the driver moves to the nearest eligible node:

$$\text{Eligible}_d = \{R_k : \alpha_{dk} = 1, R_k \text{ not yet visited}\} \cup \{N_k : \alpha_{dk} = 1, R_k \text{ visited}, N_k \text{ not yet delivered}\}$$

A restaurant R_k is eligible if it has not yet been picked up. A customer neighbourhood N_k is eligible only after its restaurant R_k has been visited. This is how pickup-before-delivery precedence is enforced, with no explicit constraint needed.

Arrival times accumulate along the route. At restaurants, the driver waits if they arrive before the food is ready: $A_d(R_k) = \max(A_d(\text{prev}) + \tau(\text{prev}, R_k, d), l_k)$. At customer neighbourhoods, a 5-minute dwell is added before departing, so departure from $N_k = A_d(N_k) + 5$.

The per-order customer wait time is then $w_k = A_d(k)(N_k) - l_k$.

Phase 3 – Feasibility Repairing

After Phase 2, the average wait $\frac{1}{k} \sum_k w_k$ and the maximum wait $\max_k w_k$ across orders is computed. If the average wait exceeds W , or if the maximum wait exceeds $W_{\max}$, the following local-search repair is applied iteratively:

1. Identify the order k^* with the highest individual wait: $k^* = \arg \max_k w_k$
2. Try reassigning k^* to every other driver $d' \neq d(k^*)$
3. For each candidate transfer, re-route only the two affected drivers and recompute the average wait
4. Accept the transfer that produces the greatest reduction in average wait
5. Repeat until the violated constraint is satisfied or no improving transfer exists

Only the two affected drivers are re-routed at each iteration, keeping each repair step fast. The maximum number of iterations is bounded by $K \times D$ to prevent infinite loops.

For the full model structure and route solution, please refer to the Appendix – Part IV.

Results

Small Instance - Benchmark Against Part III Optimal ($W = 120$)

The heuristic is first applied to `part3\_small.csv` with the same three drivers used in Part III, allowing a direct comparison against the proven MILP optimal.

Phase 1 (Clustering) processes orders in ascending release-time order. Orders 5, 1, 4, 2, and 3 are assigned in that sequence. Driver 1 (Richmond/Adelaide) is nearest to Orders 1 and 5 and receives both. Driver 2 (St. James Park) is nearest to Orders 3 and 4, and Driver 3 (Church and Wellesley) is nearest to Order 2. No repair is needed since the greedy solution already satisfies $W = 120$ with an average wait of 22.72 minutes.

Phase 2 (Routing) produces the following routes:

Driver	Orders	Route Distance	Waits (Minutes)
Driver 1 (40 km/h, Richmond/Adelaide)	1, 5	14.62 km	{1: 31.3, 5: 1.5}
Driver 2 (35 km/h, St. James Park)	3, 4	20.14 km	{3: 11.7, 4: 62.4}
Driver 3 (30 km/h, Church & Wellesley)	2	6.46 km	{2: 6.9}

Total heuristic distance: 41.2176 km | Average wait: 22.72 min | Max wait: 62.35 min | Solve time: 1.20 ms

Compared to the Part III MILP optimal:

Metric	Heuristic	MILP Optimal	Difference
Total distance (km)	41.2176	30.1836	+11.03 (+36.6%)
Average wait (min)	22.72	35.38	-12.66
Max wait (min)	62.35	100.14	-37.79
Feasible ($W=120$)?	Yes	Yes	-
Solve time	1.20 ms	~6 s	over 1,000x faster

The heuristic is 36.6% above the MILP optimal in total distance, but notably produces lower wait times on both metrics. The heuristic's average wait of 22.72 minutes is well within budget whereas the MILP's average of 35.38 minutes is closer to the constraint boundary. The distance gap arises because the MILP simultaneously optimizes all assignments globally; it discovers that giving Driver 1 four orders (rather than two) and letting Driver 2 specialize on the single proximate Order 4 produces a shorter total distance, even though the heuristic's greedy clustering cannot anticipate this.

Large Instance ($W = 120$)

The heuristic is applied to `part4\_large.csv` with 15 orders and 10 drivers, a problem size for which solving the Part III MILP directly would be infeasible within the provided time limits.

Phase 1 assigns all 15 orders across 7 of the 10 available drivers, with Drivers 5, 6, and 7 receiving no orders and remaining idle, which demonstrates that the heuristic naturally avoids unnecessary driver deployment. Phase 3 requires no repair iterations; the greedy solution already satisfies $W = 120$.

Total distance: 83.9006 km | Average wait: 81.40 min | Max wait: 176.36 min | Feasible: Yes | Solve time: 1.10 ms

The per-driver breakdown is as follows:

Driver	Orders	Route Distance	Waits (Minutes)
D1 (Richmond/Adelaide, 20 km/h)	9, 11	12.32 km	127.5, 42.0
D2 (St. James Park, 20 km/h)	4, 7, 14, 15	15.51 km	152.5, 72.0, 112.3, 50.5
D3 (Church & Wellesley, 20 km/h)	8, 10	13.45 km	176.4, 8.9
D4 (Christie, 20 km/h)	2	3.42 km	10.3
D8 (Rosedale, 20 km/h)	6, 12	5.19 km	20.6, 164.3
D9 (UofT/Harbord, 20 km/h)	5, 13	19.85 km	90.9, 64.6
D10 (Ryerson, 20 km/h)	1, 3	14.15 km	107.5, 20.9
D5, D6, D7	-	0 km	Idle

Analysis

(a) Applicability of the Heuristic

Strengths: The heuristic's most important advantage is speed. At about 1-2 milliseconds on the recorded runs for both instances, it is well over 1,000x faster than the Part III MILP for the small instance, and the MILP would be far slower still (or entirely infeasible) for the large instance. This speed is non-negotiable for real-time dispatch systems that must respond within seconds to every new order, driver cancellation, or system state change.

The heuristic is also insight-driven rather than arbitrary. Every design choice traces back directly to an observation from the Part III optimal solutions. The clustering rule replicates geographic proximity, the nearest-neighbour sequencing replicates the sweep-style routing, and the release-time ordering ensures time-sensitive orders are dispatched first. This means the algorithm can be explained and justified to operations teams without treating it as a black box.

Furthermore, the three-phase modular structure is a practical strength. Each phase can be improved independently. For example, Phase 1 could be replaced with a k-means geographic pre-clustering without touching the routing or repair logic. This makes the heuristic extensible as operational requirements evolve.

Finally, the repair phase provides a feasibility guarantee for the average wait constraint. Unlike simpler greedy methods that may silently produce infeasible solutions, this heuristic explicitly detects and corrects constraint violations through targeted order re-assignment.

Weaknesses:

Service level breach on the large instance

The large instance result requires a direct qualification: while the heuristic satisfies $W = 120$ on the average (81.40 minutes), individual orders reach maximum waits of 176.36 minutes. In a real dispatch system, any per-order guarantee — such as "delivered within two hours of food readiness" — would be violated for at least one customer in this solution. This is not a minor outlier: Driver 2's order 8 waits 176 minutes, Driver 1's order 4 waits 152 minutes, and Driver 7's order 12 waits 164 minutes. The root cause is that Phase 3 repairs average wait only. The updated heuristic includes a Phase 3b max-wait repair that targets orders exceeding $1.5 \times W = 180$ minutes. In testing, Phase 3b successfully reduces the worst-case wait on the small instance; on the large instance, the improvement depends on whether an improving re-assignment move exists. The practical mitigation for production use is a hard per-order escalation trigger: any

order whose projected wait exceeds a threshold (e.g., 150 minutes) is immediately flagged for priority re-assignment before the standard repair loop runs.

Greedy myopia

The clustering step assigns each order independently without seeing how the decision affects future assignments. The MILP discovers that giving Driver 1 four orders while Driver 2 specializing on the zero-distance Order 4 produces a globally shorter total. However, the heuristic cannot anticipate this trade-off, which explains the 36.6% distance gap on the small instance.

There is also a within-cluster sequencing cost – the nearest-neighbours rule produces routes that are locally reasonable but not globally optimal within each driver’s cluster. A short MILP sub-problem per driver, which is solvable in milliseconds for small clusters, could close this gap significantly without sacrificing the overall speed advantage.

Additionally, the repair phase has no global awareness. It transfers one order at a time and accepts the first improvement, which may leave the solution in a local minimum. Transferring two orders simultaneously or allowing a sequence of moves before evaluating improvement could produce better feasibility repairs.

(b) Online Use in Practice

In practice, ‘cluster_then_route’ would run continuously as a background process triggered by system events. The key adaptation is that each driver’s current GPS location replaces their shift-start depot as the “depot” in Phase 1. For example, a driver who has just delivered in Scarborough is not effectively based in Scarborough for the next assignment, making the proximity calculation reflect real-time geography rather than where the driver started their shift.

The operational loop works as follows. When a new order arrives, it is added to the unassigned queue. Every 30-60 seconds (a short batching window), the heuristic runs on all currently unassigned orders with all available drivers, where each driver’s position is their last known GPS location. The resulting assignments are sent to the relevant drivers’ apps. When a driver completes a delivery, their location is updated, and the next heuristic run incorporates their new position. When a driver cancels, their unassigned orders are returned to the queue and the heuristic re-runs immediately with the remaining driver pool. No model rebuild is needed, just a re-run with updated inputs.

This directly mirrors how DoorDash’s dispatch engine and Uber Eats’ real-time assignment system operate. Both platforms use fast greedy rules with local improvement, re-triggered every few seconds rather than solving a global optimization. The batching window is particularly important: without it, each order would be assigned the moment it arrives, which prevents the geographic clustering that makes routes efficient. The optimal window length is a tunable parameter. Shorter windows improve responsiveness to urgent orders while longer windows allow more batching and reduce total driver distance.

The large instance results illustrate one further consideration: 3 out of the 10 available drivers were idle under $W = 120$. In a live system, idle drivers represent wasted capacity and driver earnings. Platforms address this through surge pricing in low-demand zones and rebalancing

incentives to move drivers towards order clusters, both of which are informed by the same geographic insight that motivates the heuristic's clustering step.

Appendix

Part I Single-driver Routing

Model Structure

The route is formulated as a **flow-based Mixed Integer Program (MIP)** adapted from the standard time-indexed Traveling Salesman Problem (TSP) formulation.

Arc Indexing

The route consists of a sequence of $T = 2K$ ordered movements (arcs), indexed by

$$t = 0, 1, \dots, T - 1.$$

With K orders, there are K pickups and K deliveries, so exactly $2K$ movements are required.

Define the binary decision variable:

$$x_{ijt} = \begin{cases} 1, & \text{if the driver travels from node } i \text{ to node } j \text{ on arc } t, \\ 0, & \text{otherwise.} \end{cases}$$

Node Set

The network contains

$$n = 2K + 1$$

nodes:

- one depot,
- K restaurant nodes $R_1, \dots, R_K$,
- K neighbourhood nodes $N_1, \dots, N_K$.

A restaurant and its paired neighbourhood may correspond to the same physical location. However, they are modelled as distinct nodes to enforce the pickup–delivery relationship structurally.

Objective Function

Minimise the total travel distance:

$$\min \sum_i \sum_j \sum_t c_{ij} x_{ijt},$$

where c_{ij} denotes the travel distance (km) between nodes i and j .

Constraints

Constraint	Description
C1 — No self-loops	$x_{iit} = 0 \quad \forall i, t$
C2 — Visit each node exactly once	$\sum_i \sum_t x_{ijt} = 1 \quad \forall j \in \{\text{Restaurants} \cup \text{Neighbourhoods}\}$
C3 — Flow conservation	$\sum_i x_{ijt} = \sum_k x_{jkt+1} \quad \forall j, t = 0, \dots, T-2$
C4 — Depot departure	$\sum_j x_{0j0} = 1$
C5 — Route termination	$x_{ij,T-1} = 0$ if j is the depot or a restaurant
C6 — Pickup-before-delivery precedence	$\text{arr_step}(N_k) \geq \text{arr_step}(R_k) + 1 \quad \forall k$

Justification for C5 — End-of-Route Treatment

The case specifies that, after completing all deliveries, the driver parks in the neighbourhood where the final order is delivered. Therefore, the final movement must terminate at a neighbourhood node rather than at a restaurant or the depot.

Constraint C5 enforces this by prohibiting the final arc ($t = T - 1$) from entering either:

- the depot, or
- any restaurant node.

Since the model contains exactly $T = 2K$ arc positions and Constraint C2 requires every restaurant and neighbourhood node to be visited exactly once, the final visited node must necessarily correspond to a delivery location. Consequently, the driver cannot finish at a restaurant without violating either the arc budget or the visit constraints.

Justification for C6 — Precedence Encoding

Pickup-before-delivery precedence is enforced through weighted arc indices rather than auxiliary ordering binaries.

Specifically, the arrival step of neighbourhood node N_k must strictly exceed the arrival step of its corresponding restaurant node R_k :

$$\text{arr_step}(N_k) \geq \text{arr_step}(R_k) + 1.$$

This provides a compact linear formulation that avoids introducing $O(K^2)$ additional binary ordering constraints while still guaranteeing that each pickup occurs before its associated delivery.

Part II Time-Constrained Single-Driver Routing

Model Structure

Part II extends the arc-indexed MIP from Part I by incorporating:

continuous arrival-time variables,
restaurant release-time constraints, and
customer wait-time limits.

Sets and Parameters

Symbol	Description
K	Number of orders
$T = 2K$	Number of arc slots, indexed by $t = 0, \dots, T - 1$
$n = 2K + 1$	Total nodes: 1 depot, K restaurants, and K neighbourhoods
v	Driver speed (km/h)
c_{ij}	Distance (km) between nodes i and j
$\tau_{ij} = \frac{60c_{ij}}{v}$	Travel time (minutes) from node i to node j
l_k	Release time of order k (minutes from reference time)
M	Big- M upper bound on arrival times
W	Maximum allowable wait-time budget (minutes)

Reference Time

All timing variables are measured relative to the earliest restaurant release time across all orders.

Big- M Value

A tight but valid upper bound is:

$$M = \max_k l_k + T(\max_{ij} \tau_{ij} + 5) + 1.$$

This captures the worst-case scenario in which:

the driver first waits at the latest-releasing restaurant, and

then completes all T arcs, each requiring the maximum travel time plus the 5-minute customer dwell time.

Decision Variables

Variable	Type	Description
$x_{ijt} \in \{0, 1\}$	Binary	Equals 1 if the driver travels from node i to node j on arc t
$D_t \geq 0$	Continuous	Arrival time (minutes) at the destination node of arc t
$z_{kt} \geq 0$	Continuous	Equals D_t if arc t enters neighbourhood N_k ; otherwise 0

Objective Function

Minimise total travel distance:

$$\min \sum_i \sum_j \sum_t c_{ij} x_{ijt}.$$

Routing Constraints (C1–C7)

Constraints C1–C7 remain identical to those in Part I.

In particular:

C5 ensures the route terminates at a neighbourhood node rather than the depot or a restaurant.

C7 enforces pickup-before-delivery precedence.

Timing Constraints

T1 — Arc-Transition Clock

Arrival times accumulate travel durations:

$$D_t \geq D_{t-1} + \sum_{i \neq j} (\tau_{ij} + 5 \cdot 1[i \in \text{Neighbourhood}]) x_{ijt}, t = 1, \dots, T - 1.$$

The inequality form (rather than equality) allows implicit waiting. If food is not yet ready when the driver reaches a restaurant, Constraint T2 can increase D_t to the corresponding release time without explicitly modelling idle-time variables.

The additional 5-minute term applies only when departing from a neighbourhood node, representing the customer handoff and confirmation time after delivery.

T2 — Restaurant Release-Time Constraint

The driver cannot collect an order before it is ready:

$$D_t \geq l_k \sum_{i \neq R_k} x_{i,R_k,t}, \forall k, \forall t.$$

If arc t enters restaurant R_k , the arrival time at that step must be at least the order's release time l_k .

T3 — Big- M Linearisation for z_{kt}

These constraints isolate the arrival time associated with the delivery of order k :

$$\begin{aligned} z_{kt} &\leq D_t \forall k, t \\ z_{kt} &\leq M \sum_{i \neq N_k} x_{i,N_k,t} \forall k, t \\ z_{kt} &\geq D_t - M \left(1 - \sum_{i \neq N_k} x_{i,N_k,t} \right) \forall k, t \end{aligned}$$

When arc t enters neighbourhood N_k , the indicator term equals 1 and z_{kt} is forced to equal D_t .

Otherwise, the indicator equals 0:

the upper bound forces $z_{kt} = 0$,

and the lower bound becomes inactive.

The waiting time for order k is therefore:

$$\text{wait}_k = \sum_t z_{kt} - l_k.$$

Wait-Time Constraints

T4a — Average Wait-Time Budget

The primary formulation used in Parts II and III limits the average customer wait time:

$$\frac{1}{K} \sum_k \left(\sum_t z_{kt} - l_k \right) \leq W.$$

Equivalently:

$$\sum_k \sum_t z_{kt} - \sum_k l_k \leq KW.$$

T4b — Maximum Wait-Time Budget

An alternative formulation limits the maximum wait time for every individual order:

$$\sum_t z_{kt} - l_k \leq W, \forall k.$$

Constraint T4b is strictly stronger than T4a. Any solution feasible under T4b is automatically feasible under T4a, but the converse does not necessarily hold.

Consequently:

- the feasible region under T4b is smaller,
- the minimum feasible value of W is typically higher, and
- the optimal travel distance is weakly greater than under the average-wait formulation for the same value of W .

Part III Multi-Driver Routing and Assignment

Model Structure

Part III generalises Part II by introducing a driver index d to all variables. The model simultaneously assigns orders to drivers and determines each driver's route, replacing the sequential “assign then route” process with a single integrated MIP formulation.

Sets and Parameters

Symbol	Description
D	Number of drivers in the pool
K	Number of orders
$T = 2K$	Arc-slot budget per driver, indexed by $t = 0, \dots, T - 1$
$n = 2K + 1$	Nodes per driver: 1 depot, K restaurants, and K neighbourhoods
v_d	Speed of driver d (km/h)
c_{ij}	Distance (km) between nodes i and j
$\tau_{ij}^d = \frac{60c_{ij}}{v_d}$	Travel time (minutes) for driver d from node i to node j
l_k	Release time of order k (minutes from reference time)
M_d	Driver-specific Big- M value
W	Maximum allowable average customer wait time (minutes)

The driver-specific Big- M value is defined as:

$$M_d = \max_k l_k + T \left(\max_{ij} \tau_{ij}^d + 5 \right) + 1$$

Decision Variables

Variable	Type	Description
$x_{dijt} \in \{0, 1\}$	Binary	Equals 1 if driver d travels from node i to node j on arc t
$a_{dk} \in \{0, 1\}$	Binary	Equals 1 if order k is assigned to driver d
$D_{dt} \geq 0$	Continuous	Arrival time (minutes) of driver d at the destination node of arc t
$z_{dk t} \geq 0$	Continuous	Equals D_{dt} when arc t of driver d enters neighbourhood N_k , and 0 otherwise

Constraints

Routing constraints C1a–C6 and timing constraints T1–T3 are identical to those in Part II, now indexed by driver d .

The key additions are the integrated assignment-routing constraints.

C2a — Unique Assignment

$$\sum_d a_{dk} = 1 \forall k$$

Each order must be assigned to exactly one driver.

C2b/C2c — Assignment-Routing Linkage

$$\begin{aligned} \sum_{i,t} x_{di,R_k,t} &= a_{dk} \forall d, k \\ \sum_{i,t} x_{di,N_k,t} &= a_{dk} \forall d, k \end{aligned}$$

These constraints ensure that:

- the same driver completes both pickup and delivery for order k , and
- a driver visits the restaurant and neighbourhood nodes if and only if the order is assigned to them.

T4 — Average Wait-Time Constraint

$$\frac{1}{K} \sum_k \left(\sum_d \sum_t z_{dkt} - l_k \right) \leq W$$

This limits the average customer wait time across all orders.

Wait-Time Worked Example Illustration

The multi-driver wait-time expression is:

$$\text{wait}_k = \sum_d \sum_t z_{dkt} - l_k$$

Although this appears more complex than the single-driver version, the logic is identical. Since Constraint C2a guarantees that each order is assigned to exactly one driver, all other z_{dkt} values are zero. The expression therefore reduces to the arrival time of the assigned driver at neighbourhood N_k .

Example Setup

Assume:

- 2 orders,
- 2 drivers,
- all drivers travel at 40 km/h.

Orders

Order	Restaurant	Neighbourhood	Release Time
1	R_1 = Rosedale	N_1 = Bay Street	0 min
2	R_2 = St James Park	N_2 = Beaches	36 min

Driver Assignments

- Driver 0 (starting from Richmond/Adelaide) is assigned Order 1.
- Driver 1 (starting from St James Park) is assigned Order 2.

Driver 0 Route

Route:

Depot $\rightarrow R_1 \rightarrow N_1$

Travel details:

- Depot to R_1 : 2.1 km = 3.2 min
- R_1 to N_1 : 0.8 km = 1.2 min

Arrival at N_1 :

$$3.2 + 1.2 = 4.4$$

Therefore:

$$\text{wait}_1 = 4.4 - 0 = 4.4 \text{ min}$$

Driver 1 Route

Route:

Depot $\rightarrow R_2 \rightarrow N_2$

Travel details:

- Depot to R_2 : 0 km = 0 min
- R_2 to N_2 : 7.1 km = 10.7 min

Since the food is not ready until 36 minutes, the driver waits at the restaurant.

Arrival at N_2 :

$$\max(0, 36) + 10.7 = 46.7$$

Therefore:

$$\text{wait}_2 = 46.7 - 36 = 10.7 \text{ min}$$

Final Result

Average customer wait time:

$$\frac{4.4 + 10.7}{2} = 7.55 \text{ min}$$

Therefore, the solution is feasible whenever:

$$W \geq 7.55$$

Part IV Cluster-then-route greedy heuristic

Sets and Parameters

Symbol	Description
K	Number of orders
D	Number of drivers
v_d	Speed of driver d (km/h)
$c(a, b)$	Haversine distance (km) between locations a and b
$\tau(a, b, d) = 60 \cdot c(a, b) / v_d$	Travel time (minutes) for driver d from a to b

l_k	Release time of order k (minutes from reference time, i.e. earliest release across all orders)
W	Maximum allowable average customer wait time (minutes)
$depot_d$	Starting location of driver d
R_k	Restaurant node for order k
N_k	Neighbourhood (delivery) node for order k

Decision Variables

Variable	Type	Description
$\alpha_{dk} \in \{0,1\}$	Binary	1 if order k is assigned to driver d, 0 otherwise
σ_d	Ordered list	Sequence of nodes visited by driver d (route), constructed greedily
$A_{d(v)}$	Continuous	Arrival time (minutes from reference) of driver d at node v

Objective

Minimise total distance traversed across all drivers:

$$\min \sum_{d=1}^D \sum_{consecutive(u,v) \in \sigma_d} c(u,v)$$

Constraints

Phase 1 – Assignment (Clustering):

Each order is assigned to exactly one driver:

$$\sum_{d=1}^D \alpha_{dk} = 1, \quad \forall k \in \{1, \dots, K\}$$

Orders are processed in ascending release-time order, and each order k is greedily assigned to the nearest driver by depot-to-restaurant distance:

$$\alpha_{d^*k} = 1, \quad d^* = \operatorname{argmin}_d c(depot_d, R_k)$$

Phase 2 – Routing (Nearest-Neighbour per Driver):

At each step, the driver moves to the nearest eligible node. A restaurant R_k is eligible if assigned to this driver and not yet visited. A neighbourhood N_k is eligible if R_k has been visited (food picked up) but N_k has not yet been visited (delivery pending):

$$\text{Eligible } d(t) = \{R_k : \alpha_{dk} = 1, R_k \text{ not yet visited}\} \cup \{N_k : \alpha_{dk} = 1, R_k \text{ visited, } N_k \text{ not yet visited}\}$$

The next node visited is: $v^* = \operatorname{argmin}_{v \in \text{Eligible } d(t)} c(\text{current location of } d, v)$

Arrival time accumulation:

$$A_{d(R_k)} = \max(A_{d(\text{prev})} + \tau(\text{prev}, R_k, d), l_k)$$

(The driver waits at the restaurant if arriving before the food is ready.)

$$A_{d(N_k)} = A_{d(\text{prev})} + \tau(\text{prev}, N_k, d)$$

After each neighbourhood visit, a fixed 5-minute customer pickup dwell is added before departing:

$$\text{departure from } N_k = A_{d(N_k)} + 5$$

Precedence (pickup before delivery):

Enforced structurally by the eligibility rule: N_k only becomes eligible after R_k has been visited.

Phase 3 – Feasibility Repair:

The per-order wait time is defined as:

$$W_k = A_{d_k(N_k)} - l_k$$

where d_k denotes the driver assigned to order k . The wait constraint is:

$$\text{Average wait constraint: } \frac{1}{K} \sum_{k=1}^K W_k \leq W$$

and

$$\text{Maximum wait constraint: } \max_k W_k \leq K_{\max}$$

If either constraint is violated after Phase 2, a local-search repair is applied iteratively:

1. Identify the order k^* with the largest wait time: $k^* = \operatorname{argmax}_k W_k$.
2. Try re-assigning k^* to every other driver $d' \neq d(k^*)$.
3. For each candidate re-assignment, re-route only the two affected drivers and recompute average wait.
4. Accept the move yielding the greatest reduction in average wait.
5. Repeat until the violated constraint is satisfied or no improving move exists.

Part IV Solution Routing

Routing Solution – Small Instance (5 Orders, 3 Drivers)

Summary: Total Distance = 41.22 km | Avg Wait = 22.72 min | Max Wait = 62.35 min | W = 120 Feasible

Driver 0 (Speed: 40 km/h | Depot: DT (R/A/K)) — Orders: [1, 5]

Step	Move To	From	To	Distance (km)	Arrival Time (min)
1	Restaurant	DT (R/A/K)	DT (Central Bay St)	0.787	1.18
2	Restaurant	DT (Central Bay St)	DT (Kensington/Chinatown)	0.983	7.00
3	Neighbourhood	DT (Kensington/Chinatown)	DT (Central Bay St)	0.983	8.47
4	Neighbourhood	DT (Central Bay St)	North York (A/W/D)	11.865	31.27

Driver 1 (Speed: 35 km/h | Depot: DT (St. James Park)) — Orders: [3, 4]

Step	Move To	From	To	Distance (km)	Arrival Time (min)
1	Restaurant	DT (St. James Park)	DT (St. James Park)	0.000	36.00
2	Restaurant	DT (St. James Park)	DT (Ryerson)	0.691	60.00
3	Neighbourhood	DT (Ryerson)	York (Fairbank/Oakwood)	6.807	71.67
4	Neighbourhood	York (Fairbank/Oakwood)	East Toronto (The Beaches)	12.647	98.35

Driver 2 (Speed: 30 km/h | Depot: DT (Church & Wellesley)) — Orders: [2]

Step	Move To	From	To	Distance (km)	Arrival Time (min)
1	Restaurant	DT (Church & Wellesley)	DT (Christie)	3.031	32.00
2	Neighbourhood	DT (Christie)	West Toronto (Brockton/Parkdale/Exhib)	3.424	38.85

Routing Solution – Large Instance (15 Orders, 10 Drivers)

Summary: Total Distance = 83.90 km | Avg Wait = 81.40 min | Max Wait = 176.36 min | W = 120 Feasible

Driver 0 (Speed: 20 km/h | Depot: DT (R/A/K)) — Orders: [9, 11]

Step	Move To	From	To	Distance (km)	Arrival Time (min)
1	Restaurant	DT (R/A/K)	DT (R/A/K)	0.000	135.00
2	Restaurant	DT (R/A/K)	DT (Underground City)	0.092	135.28
3	Neighbourhood	DT (Underground City)	Central Toronto (Summerhill W/Rathnelly)	4.413	148.52
4	Neighbourhood	Central Toronto (Summerhill W/Rathnelly)	East York (Parkview Hill/Woodbine)	7.815	176.96

Driver 1 (Speed: 20 km/h | Depot: DT (St. James Park)) — Orders: [4, 7, 14, 15]

Step	Move To	From	To	Distance (km)	Arrival Time (min)
1	Restaurant	DT (St. James Park)	DT (St. James Park)	0.000	53.00
2	Restaurant	DT (St. James Park)	DT (St. James Park)	0.000	56.00
3	Restaurant	DT (St. James Park)	DT (St. James Park)	0.000	144.00
4	Restaurant	DT (St. James Park)	DT (Regent Pk/Port)	1.145	147.44
5	Neighbourhood	DT (Regent Pk/Port)	DT (TD Centre/Design)	1.851	152.99
6	Neighbourhood	DT (TD Centre/Design)	Central Toronto (Annex/N.Midtown)	3.424	168.26
7	Neighbourhood	Central Toronto (Annex/N.Midtown)	East Toronto (India Bazaar/Beaches W)	7.094	194.54
8	Neighbourhood	East Toronto (India Bazaar/Beaches W)	East Toronto (The Beaches)	1.997	205.53

Driver 2 (Speed: 20 km/h | Depot: DT (Church & Wellesley)) — Orders: [8, 10]

Step	Move To	From	To	Distance (km)	Arrival Time (min)
1	Restaurant	DT (Church & Wellesley)	DT (Church & Wellesley)	0.000	141.00
2	Restaurant	DT (Church & Wellesley)	DT (St. James Town/Cabbagetown)	1.176	144.53
3	Neighbourhood	DT (St. James Town/Cabbagetown)	East Toronto (Danforth W/Riverdale)	1.796	149.92
4	Neighbourhood	East Toronto (Danforth W/Riverdale)	York (Runnymede/Junction N)	10.481	186.36

Driver 3 (Speed: 20 km/h | Depot: DT (Christie)) — Orders: [2]

Step	Move To	From	To	Distance (km)	Arrival Time (min)
1	Restaurant	DT (Christie)	DT (Christie)	0.000	49.00
2	Neighbourhood	DT (Christie)	West Toronto (Brockton/Parkdale/Exhib)	3.424	59.27

Driver 4 (Speed: 20 km/h | Depot: —) — No assigned orders**Driver 5 (Speed: 20 km/h | Depot: —) — No assigned orders****Driver 6 (Speed: 20 km/h | Depot: —) — No assigned orders****Driver 7 (Speed: 20 km/h | Depot: DT (Rosedale)) — Orders: [6, 12]**

Step	Move To	From	To	Distance (km)	Arrival Time (min)
1	Restaurant	DT (Rosedale)	DT (Rosedale)	0.000	155.00
2	Restaurant	DT (Rosedale)	DT (Rosedale)	0.000	155.00
3	Neighbourhood	DT (Rosedale)	DT (Central Bay St)	3.106	164.32
4	Neighbourhood	DT (Central Bay St)	DT (CN Tower / King and Spadina /	2.084	175.57

Driver 8 (Speed: 20 km/h | Depot: DT (UofT/Harbord)) — Orders: [5, 13]

Step	Move To	From	To	Distance (km)	Arrival Time (min)
1	Restaurant	DT (UofT/Harbord)	DT (UofT/Harbord)	0.000	109.00

2	Restaurant	DT (UofT/Harbord)	DT (Kensington/Chinatown)	0.981	111.94
3	Neighbourhood	DT (Kensington/Chinatown)	DT (Central Bay St)	0.983	114.89
4	Neighbourhood	DT (Central Bay St)	North York (Humber Summit)	17.889	173.56

Driver 9 (Speed: 20 km/h | Depot: DT (Ryerson)) — Orders: [1, 3]

Step	Move To	From	To	Distance (km)	Arrival Time (min)
1	Restaurant	DT (Ryerson)	DT (Ryerson)	0.000	77.00
2	Restaurant	DT (Ryerson)	DT (Central Bay St)	0.626	78.88
3	Neighbourhood	DT (Central Bay St)	York (Fairbank/Oakwood)	6.335	97.88
4	Neighbourhood	York (Fairbank/Oakwood)	North York (A/W/D)	7.189	124.45